Bio Bloom Software Requirements Specification (SRS) v2.0

Updated to the final implementation

Item	Value
Project	Bio Bloom – Artisanal Food E-Commerce Platform
Course	SENG 34213 System Development Project
Department	Department of Software Engineering, University of Kelaniya
Document	Software Requirements Specification (SRS)
Version	2.0
Date	25 October 2026
Based on	SRS v01 (2026/03/21), SDS v01 (2026/04/07), backlog `docs/development-issues.md`, ADR-001 to ADR-014, and the code in this repository
Team	I.G.R. Laksara (SE/2021/028), Prageeth Ekanayake (SE/2021/049), Sanduni Ayeshika (SE/2021/039), Chamindu Dilshan (SE/2021/058), Ovindu Pathirana (SE/2021/059)

Version history

Version	Date	Summary
01	2026/03/21	Initial SRS (the baseline named in `docs/development-issues.md`). The file is not in the repository.
2.0	25 October 2026	Rebuilt from the SDS, the backlog and the code. Requirements renumbered FR001 to FR030 and NFR001 to NFR020, each with a verified implementation status.

Team and responsibilities

Member	GitHub	Area
Ovindu Pathirana	`ovindu17`	Backend, database, cron jobs, security, DevOps
Prageeth Ekanayake	`Prageethek`	Backend, database, cron jobs, security, DevOps
Chamindu Dilshan	`Chamindu284`	Frontend
Sanduni Ayeshika	`sanduni4`	Frontend
I.G.R. Laksara	`RavinduLaksara`	Mobile (Flutter delivery app)

Table of contents

1. Introduction
    1. Purpose

    2. Scope
    3. Definitions, acronyms and abbreviations
    4. References
    5. How to read this document
    6. Changes from v01
2. Overall description
    1. Product perspective
    2. Product functions
    3. User classes and characteristics
    4. System roles and permissions
    5. Operating environment
    6. Design and implementation constraints
    7. Assumptions and dependencies
3. External interface requirements
4. Functional requirements
5. Non-functional requirements
6. Use-case overview
7. Traceability matrix
8. Open gaps and follow-up checklist
9. Appendix A: Data model summary
10. Appendix B: Business rules enforced in code

---

1. Introduction

1.1 Purpose

This document states what Bio Bloom must do and how well it must do it. Version 2.0 replaces SRS v01. It matches the requirements to the system that the team actually built, so readers can see three things for every requirement:

- where it came from (SDS section, backlog issue `DDP-#NN` or ADR);
- which code implements it (repository path);
- whether it is **Implemented**, **Partial** or **Not implemented** today.

The readers are the project supervisor, the client (the artisanal food business), the five team members and future maintainers.

1.2 Scope

Bio Bloom is an online shop for packaged, homemade Sri Lankan foods such as jams, chilli sauces and pickles. SDS v01 described one vendor. The built system is a **multi-seller marketplace** (ADR-002): approved artisan sellers list their own products, and Bio Bloom administrators run the platform.

The system has five client-facing parts and one API:

Part	Technology	Path	Main users
Customer storefront	Next.js 14 App Router, next-intl	`apps/storefront`	Shoppers and subscribers
Admin console	Next.js 14 App Router	`apps/admin`	Administrators, fulfilment staff
Seller portal	Next.js 14 App Router	`apps/seller`	Approved sellers, applicants
Delivery app	Flutter 3	`apps/delivery`	Delivery riders
REST API	NestJS 10, Prisma 6, Node 20	`apps/api`	All clients
Shared packages	TypeScript	`packages/prisma`, `packages/ui`, `packages/shared-types`	Developers

The four business problems from SDS §1.2 remain the core of the product:

1. **Food safety:** FIFO batch tracking with expiry enforcement.
2. **Recurring revenue:** Subscribe & Save with automatic billing.
3. **Logistics:** shipping fees based on real weight (glass jars are heavy).
4. **Product flexibility:** variants (jar sizes) and gift-box bundles.

Out of scope for this release: seller payout settlement, customer favourites, SMS and in-app notifications, a working newsletter subscription, live GPS tracking, and the production AWS environment. Section 8 lists each gap.

Where status is measured. Status in this document is checked against the integrated code in this repository (`biobloom`). The team's GitHub repositories (`seng34213-biobloom/backend`, `frontend`, `mobile`) carry the same work through pull requests; some of it is still open there. Section 4.10 lists those differences.

1.3 Definitions, acronyms and abbreviations

Term	Meaning
ADR	Architecture Decision Record, stored in `docs/adr/`
Batch	One production run of a variant, with manufacture date, expiry date and quantity (`ProductBatch`)
Bundle	A gift box made of several variants (`Bundle`, `BundleItem`)
CSRF	Cross-Site Request Forgery. Bio Bloom uses the double-submit cookie pattern
DDP-#NN	A development issue in `docs/development-issues.md` (40 issues, Sprints 5 to 8)
FCM	Firebase Cloud Messaging, used for push notifications to riders
FIFO	First-In, First-Out. Here: always sell from the batch with the earliest expiry date
JTI	JWT ID. Stored in Redis on logout so the token can no longer be used
JWT	JSON Web Token, signed with HS256
LKR	Sri Lankan rupee, the default currency
Mock mode	`PAYMENTS_MODE=mock`: a simulated hosted checkout that runs the real webhook code path (ADR-012)

Term	Meaning
POD	Proof of delivery: a photo, plus GPS when the phone can provide it
RBAC	Role-Based Access Control
Rider	A delivery team member who uses the Flutter app (role `DELIVERY`)
S&S	Subscribe & Save, the recurring-order feature
Seller	An artisan business approved to sell on Bio Bloom (role `SELLER`)
SKU	Stock Keeping Unit, one per variant
Variant	A size or flavour of a product, with its own SKU, price and weight (`ProductVariant`)

1.4 References

ID	Document
R1	Bio Bloom System Design Specification (SDS) v01, 2026/04/07 (`Bio Bloom SDS Final (1).pdf`)
R2	Development issues, Sprints 5 to 8: `docs/development-issues.md`
R3	Architecture decisions ADR-001 to ADR-014: `docs/adr/`
R4	API reference: `docs/api-reference.md`
R5	WCAG 2.1 AA audit: `docs/wcag-audit.md`
R6	Load-test plan: `infra/load-test/README.md`
R7	`README.md`, `CHANGELOG.md`, `docs/local-preview.md`
R8	IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
R9	W3C, Web Content Accessibility Guidelines (WCAG) 2.1
R10	OWASP Top Ten; PCI DSS v4.0

1.5 How to read this document

Requirement IDs. SRS v01 is not in the repository. The code and docs already cite eight IDs: `FR001`, `FR002`, `FR011`, `FR012`, `FR015`, `FR021`, `FR029` and `NFR009`. Version 2.0 keeps those eight with the meaning the code gives them. Every other ID is assigned here and replaces the `FR-TBC` and `NFR-TBC` placeholders in `docs/development-issues.md`.

> **Note on FR029.** ADR-002 links FR029 to administrator approval of sellers. The code (`apps/api/src/sellers/seller-portal.service.ts`, `apps/seller/app/(portal)/dashboard/page.tsx`) uses FR029 for the seller dashboard. Version 2.0 follows the code: FR029 is the seller dashboard and earnings, and seller approval is FR002.

Priority is the backlog label of the source issue: `P1-Blocker`, `P2-Critical`, `P3-Major`, `P4-Minor`. "Not set" means no backlog issue exists.

Functional status:

Status	Meaning
Implemented	API and user interface exist and meet the requirement as written here
Partial	Some of the requirement is built; the missing part is named
Not implemented	No working code. A schema model or a placeholder alone does not count

Non-functional status: Met, Partial, Not met, or Not measured (a target exists but nobody has run the measurement).

1.6 Changes from v01

#	Topic	SRS v01 / SDS v01	Version 2.0 (as built)	Evidence
1	Business model	Single vendor (SDS §1.2)	Multi-seller marketplace: sellers apply, an admin approves, each product belongs to one seller	ADR-002; `Seller` in `packages/prisma/prisma/schema.prisma`; `apps/seller`
2	Roles	Three roles: CUSTOMER, ADMIN, FULFILMENT_STAFF (SDS §7.2)	Five roles: CUSTOMER, ADMIN, FULFILMENT_STAFF, SELLER, DELIVERY	`enum Role` in `schema.prisma`; `packages/shared-types/src/index.ts`
3	Identity storage	Customer and AdminUser tables	Four actor tables: `customer`, `admin_user` (ADMIN or FULFILMENT_STAFF), `seller`, `delivery_user`. One login; JWT carries `actorType`	ADR-005; `apps/api/src/auth/auth.service.ts`
4	Data model	14 entities (SDS §3.2)	25 Prisma models. Added: ProductImage, Seller, Payout, PromoCode, RefundRequest, FavoriteProduct, AuditLog, DeliveryUser, DeliveryAssignment, SupportTicket, CartItem	`schema.prisma`; Appendix A
5	Notifications	SRS mentioned SMS and in-app messages; SDS lists SendGrid email	Email only for customers, sellers and admins; FCM push only for riders. SMS and in-app messages dropped	ADR-003
6	Email transport	SendGrid v7 API (SDS §2.3)	SMTP through nodemailer: MailHog locally, SendGrid's SMTP relay in deployed environments	`apps/api/src/notifications/mailer.service.ts`
7	Stock reservation	Order created and stock decremented on payment confirmation (SDS §5.3)	Stock is reserved at checkout in `PENDING_PAYMENT`. Unpaid orders are cancelled and restocked after 30 minutes	ADR-007, ADR-011; `apps/api/src/orders/stale-order-cron.service.ts`
8	Payments in development	Gateway sandboxes	Mock mode by default outside production; live Stripe and PayHere otherwise	ADR-012; `apps/api/src/payments/payments.config.ts`

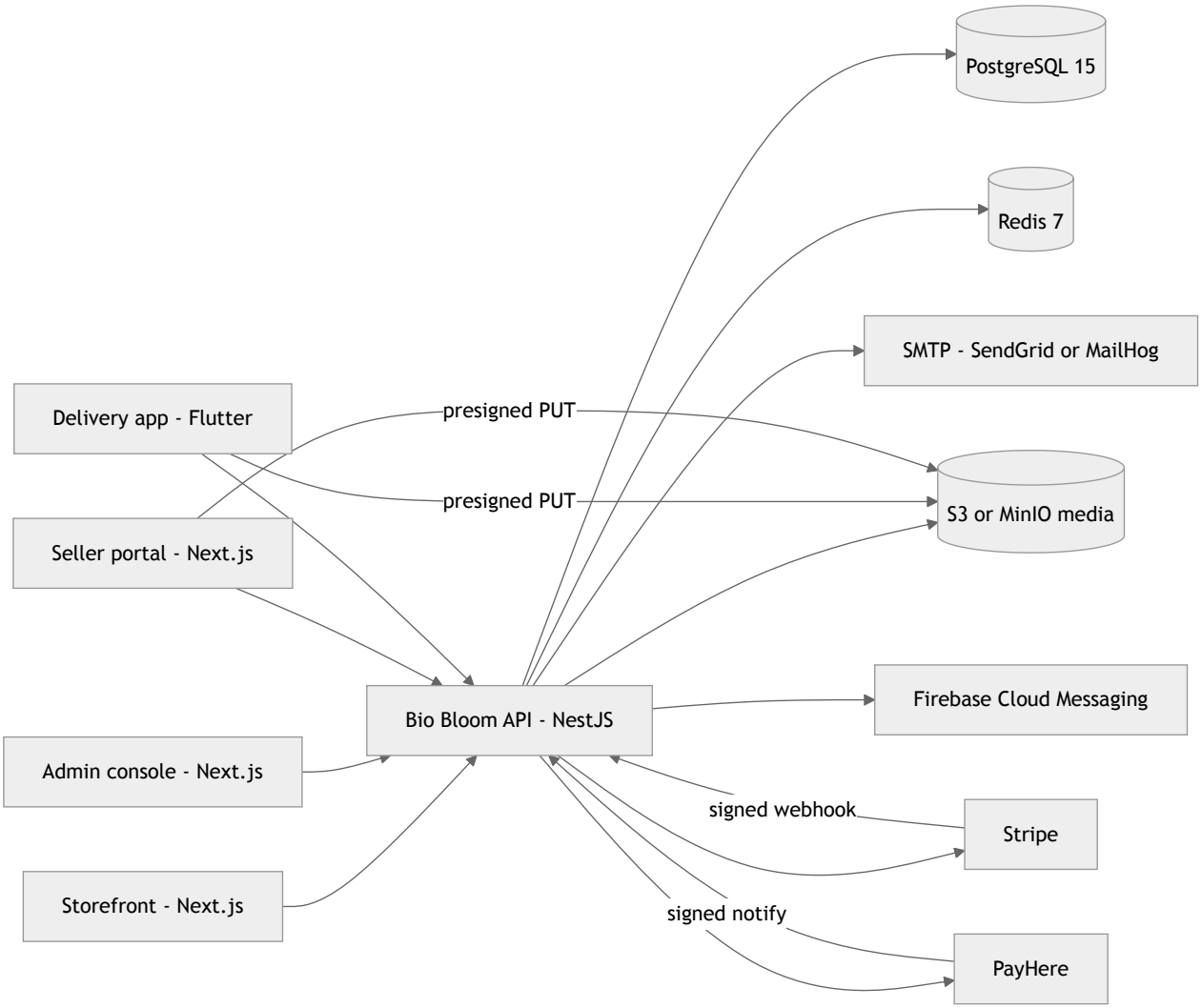
#	Topic	SRS v01 / SDS v01	Version 2.0 (as built)	Evidence
9	S&S frequency	"every 30 or 60 days" (SDS §5.5)	14, 30 or 60 days; fixed 10% discount locked at sign-up	`apps/api/src/subscriptions/dto/`, `subscriptions.service.ts` (`DEFAULT_DISCOUNT_RATE`)
10	Cron time zone	"06:00 daily"	06:00 UTC (11:30 Sri Lanka time); all date arithmetic in UTC; one instance per job per day through a Redis lock	ADR-008; `apps/api/src/main.ts`; `common/locks/cron-lock.service.ts`
11	Image uploads	Validated upload to S3 (SDS §7.4)	Presigned S3 PUT, then magic-byte check on attach; 5 MB cap; JPEG, PNG, WebP, HEIC	ADR-014; `apps/api/src/uploads/uploads.service.ts`
12	Failed deliveries	Not covered	Failure needs a reason; staff re-open the same assignment with an attempt counter	ADR-013; `apps/api/src/delivery/delivery.service.ts`
13	Languages	Not specified in the SDS	Storefront and API error messages in English and Sinhala, locale held in a cookie	ADR-010; `apps/storefront/i18n/`; `apps/api/src/i18n/`
14	New customer features	Not in the SDS	Promo codes, refunds (full and partial), moderated reviews, support tickets	`apps/api/src/promo-codes`, `refunds`, `reviews`, `support`
15	Deployment	AWS ECS Fargate, RDS, ElastiCache, S3, CloudFront (SDS §8)	Staging runs on Render (API, PostgreSQL 15, Key Value) and Vercel (storefront). The AWS Terraform in `infra/terraform` is written but not applied	Team backend PR #51; `infra/terraform/*.tf`
16	Versions	Prisma v5+, Next.js 14+	Prisma `^6.0.0`, Next.js `14.2.5`, NestJS `^10.4.0`	`package.json` files

---

2. Overall description

2.1 Product perspective

Bio Bloom is a new, self-contained product. All clients call one REST API over HTTPS. The API owns all business rules.



2.2 Product functions

- Browse and search a stock-aware catalogue of products and gift boxes.
- Keep a cart, see the weight-based shipping fee, apply a promo code, and pay through a hosted checkout.
- Subscribe to a variant, then pause, resume, skip or cancel it; the system bills it automatically.
- Track orders, ask for refunds, write reviews and raise support tickets.
- Sellers apply, then manage their own products, variants, images, batches and bundles, and see their sales.
- Administrators approve sellers, manage the catalogue and batches, run fulfilment, assign riders, moderate reviews, handle refunds and tickets, and see business figures.
- Riders see their assignments, update status with GPS, capture proof of delivery and get push alerts.
- Daily jobs expire old batches and alert admins, bill subscriptions, and cancel unpaid orders.

2.3 User classes and characteristics

User class	Description	Technical skill	Client
Visitor	Anyone browsing without an account	Low	Storefront

User class	Description	Technical skill	Client
Customer	Registered shopper. Buys once or subscribes. May use English or Sinhala	Low	Storefront
Seller applicant	An artisan business that has applied but is not approved yet. Cannot sign in	Low to medium	Seller portal (onboarding)
Seller	Approved artisan business managing its own listings and stock	Medium	Seller portal
Administrator	Bio Bloom operator with full control of the platform	Medium	Admin console
Fulfilment staff	Packs and ships orders and assigns riders	Low to medium	Admin console
Rider	Delivers orders with an Android or iOS phone, often with weak connectivity	Low	Delivery app
Scheduler (system)	In-process cron jobs: billing, expiry sweep, stale-order cleanup	Not applicable	API

2.4 System roles and permissions

The `Role` enum has five values: `CUSTOMER`, `ADMIN`, `FULFILMENT_STAFF`, `SELLER`, `DELIVERY` (`packages/prisma/prisma/schema.prisma`). Three global guards run on every request in this order (`apps/api/src/app.module.ts`): `JwtAuthGuard`, `UserThrottlerGuard`, `RolesGuard`. Routes marked `@Public()` skip authentication. `@Roles(...)` on a controller or method limits it to the listed roles (`apps/api/src/auth/guards/roles.guard.ts`). Services add ownership checks on top, for example "Sellers can only manage their own products" (`apps/api/src/products/products-admin.service.ts`).

Sign-in rules (`apps/api/src/auth/auth.service.ts`): a seller can sign in only when `status = APPROVED`; admin, staff and rider accounts must be `active`. Only customers can self-register.

Capability	Public	CUSTOMER	SELLER	FULFILMENT_STAFF	ADMIN	DELIVERY
Browse products, bundles, approved reviews	Yes	Yes	Yes	Yes	Yes	Yes
Register (customer), apply as seller	Yes					
Sign in, sign out, read own profile (`/api/auth/*`)		Yes	Yes	Yes	Yes	Yes
Addresses, cart, checkout, own orders, cancel unpaid order		Yes				
Subscriptions: create, pause, resume, skip next, cancel		Yes				
Save a payment method; simulate payment (mock mode only)		Yes				
Request a refund; post a review; open or reopen a support ticket		Yes				
Manage products, variants, images, batches, bundles			Own only		All	

Capability	Public	CUSTOMER	SELLER	FULFILMENT_STAFF	ADMIN	DELIVERY
Seller profile, dashboard figures, own sales lines (`/api/seller/*`)			Yes			
Admin order list and detail				Yes	Yes	
Cancel a paid order (with refund and restock)					Yes	
Fulfilment queue: packing, shipped, delivered				Yes	Yes	
Assign or re-assign a rider; view assignments; list riders				Yes	Yes	
Create and edit riders and staff accounts					Yes	
Approve, reject, suspend, reinstate sellers					Yes	
Moderate reviews; approve or reject refunds; resolve or close tickets					Yes	
Promo codes, shipping-rate brackets					Yes	
Subscription oversight; manual billing run (not in production)					Yes	
Analytics: KPIs, revenue series, top-selling products					Yes	
Own assignments: status with GPS, proof photo, FCM token						Yes

What each role can do, in short:

- **CUSTOMER.** Controllers `addresses`, `cart`, `orders` (customer part), `subscriptions`, `payments`, plus customer routes in `refunds`, `reviews`, `support`. Every customer resource is checked for ownership (`apps/api/test/ownership.e2e-spec.ts`).
- **ADMIN.** Full control. Only role allowed on `admin/analytics`, `admin/sellers`, `admin/promo-codes`, `admin/shipping-rates`, `admin/staff`, `admin/subscriptions`, admin refund, review and support routes, rider create and edit, and `POST admin/orders/:id/cancel`.
- **FULFILMENT_STAFF.** Shares `admin/orders` (read), `admin/fulfilment`, the delivery-assignment routes in `delivery-admin.controller.ts` and `GET admin/riders` with ADMIN. Cannot cancel orders, manage the catalogue, or see analytics.
- **SELLER.** Shares `admin/products`, `admin/products/:id/batches`, `admin/bundles` and `POST uploads/product-image` with ADMIN, but services restrict every call to the seller's own products. Has its own `seller/me`, `seller/summary`, `seller/orders`.
- **DELIVERY.** Only `delivery/*` routes and `POST uploads/proof-photo`, and only for assignments that belong to the rider.

2.5 Operating environment

Component	Environment
API	Node.js 20, NestJS 10, process time zone forced to UTC
Database	PostgreSQL 15 with the pgcrypto extension
Cache and locks	Redis 7 (JWT blocklist, shipping-rate cache, rate-limit counters, cron locks)
Media	S3-compatible storage (MinIO locally)
Web clients	Current desktop and mobile browsers; layouts checked at 320 px width (`docs/wcag-audit.md`)
Mobile client	Android 10 (API 29) or later; iOS 13 or later (NFR009)
Local	Docker Compose: PostgreSQL on host port 5433, Redis, MailHog, MinIO, API and three web apps (`docker-compose.yml`)
Staging	Render: API `https://biobloom-api.onrender.com`, PostgreSQL 15, Key Value. Vercel: storefront `https://biobloom-storefront.vercel.app`

2.6 Design and implementation constraints

- One pnpm monorepo for TypeScript code; the Flutter app sits beside it with its own toolchain (ADR-001).
- PostgreSQL and Prisma. Raw SQL is allowed only for pgcrypto calls, the FIFO lock query and atomic counters, and must use tagged templates with parameters (ADR-004, ADR-007).
- No card data may be stored or logged. Payments use hosted checkout only (ADR-009, SDS §7.3).
- Every daily job and date comparison uses UTC (ADR-008).
- Money is held as `Decimal(12,2)` and serialised as two-decimal strings (ADR-004).
- The API must refuse to start in production with missing or development-value secrets (`apps/api/src/config/env.validation.ts`).
- Academic constraints: four sprints (Sprints 5 to 8) and a backlog estimate of 414 hours (`docs/development-issues.md`).

2.7 Assumptions and dependencies

- Customers have an email address; email is the only customer notification channel.
- Riders have a smartphone with a camera. GPS is used when the rider allows it; a status update is still sent without it (`apps/delivery/lib/app/location_service.dart`).
- Stripe, PayHere, SendGrid, Firebase and an S3 bucket are available and configured in deployed environments. Without them, mock payments, MailHog, MinIO and "push skipped" logging keep local preview working (ADR-003, ADR-012).
- Firebase files (`google-services.json`, `GoogleService-Info.plist`, service-account JSON) are kept outside git.
- **Known staging issue:** the auth cookie is `SameSite=Strict`, so sign-in from the Vercel storefront to the Render API fails because the request is cross-site. This is tracked in team frontend issue #20, with fix PR #21 (API proxy through the storefront) still open.

---

3. External interface requirements

3.1 User interfaces

Client	Screens (path)
Storefront	Home, products, product detail, bundles, cart, checkout, payment page (mock), checkout complete, login and register, account: orders, order detail, subscriptions, addresses, support (`apps/storefront/app/`)
Admin console	Dashboard, products, product batches, bundles, sellers, orders, fulfilment, deliveries, riders, staff, refund requests, reviews, support, promo codes, shipping rates, subscriptions (`apps/admin/app/`)
Seller portal	Landing, onboarding (application), login, dashboard, products, product batches, bundles, orders, profile (`apps/seller/app/`)
Delivery app	Login, assignments feed, proof of delivery (`apps/delivery/lib/screens/`)

All web apps share the Tailwind preset and components in `packages/ui`, including a global `:focus-visible` outline.

3.2 Software interfaces

System	Purpose	Interface	Code
Stripe	Card payments, saved cards for S&S, refunds	Checkout Session redirect; setup session and SetupIntent; off-session PaymentIntent with idempotency key; Refunds; webhook `POST /api/webhooks/stripe` verified with `Stripe-Signature` (HMAC-SHA256, timestamp tolerance)	`apps/api/src/payments/gateways/stripe.gateway.ts`
PayHere	LKR payments, recurring token, refunds	Signed form post (`FORM_POST`); notify `POST /api/webhooks/payhere` verified by `md5sig`; Merchant API `/merchant/v1/payment/charge` and `/merchant/v1/payment/refund`	`apps/api/src/payments/gateways/payhere.gateway.ts`
Mock gateway	Payments without accounts (development, CI)	Storefront page `/checkout/pay`; `POST /api/payments/:paymentId/simulate` (404 in live mode)	`apps/api/src/payments/gateways/mock.gateway.ts`
SendGrid / SMTP	Transactional email	SMTP via nodemailer: SendGrid relay (TLS, port 465) in deployed environments, MailHog on port 1025 locally. Header `X-Biobloom-Template`	`apps/api/src/notifications/mailer.service.ts`, `templates.config.ts`
Amazon S3 (MinIO locally)	Product images and proof-of-delivery photos	Presigned PUT from `POST /api/uploads/product-image` and `/proof-photo`; server reads first bytes to verify the type	`apps/api/src/uploads/uploads.service.ts`

System	Purpose	Interface	Code
Firebase Cloud Messaging	Push to riders on new assignment	Firebase Admin SDK with `FIREBASE_SERVICE_ACCOUNT_JSON`; token registered by `POST /api/delivery/fcm-token`	`apps/api/src/notifications/push.service.ts`, `apps/delivery/lib/app/push_service.dart`
PostgreSQL 15	System of record	Prisma client; pgcrypto `pgp_sym_encrypt` and `pgp_sym_decrypt` for payment tokens	`packages/prisma`, `apps/api/src/subscriptions/token-cipher.service.ts`
Redis 7	JWT blocklist, shipping cache, throttling, cron locks	ioredis	`apps/api/src/redis/redis.service.ts`
Google Maps	Turn-by-turn navigation for riders	Opens the address in the maps app through an `https` link	`apps/delivery/lib/screens/assignments_feed_screen.dart`

3.3 Communication interfaces

- REST over HTTPS with JSON. All routes sit under `/api`; `/api/v1/*` is accepted as an alias (`apps/api/src/common/middleware/http-request.middleware.ts`).
- Browsers send the JWT in the `auth_token` cookie (`HttpOnly`, `SameSite=Strict`, `Secure` in production). The rider app sends `Authorization: Bearer <token>` (ADR-006).
- Every non-GET request must send the `X-CSRF-Token` header matching the `csrf-token` cookie. Webhooks are exempt because their signatures are checked (`apps/api/src/common/middleware/csrf.middleware.ts`).
- Validation errors return HTTP 422. Error messages follow `Accept-Language` (`en` or `si`) (`apps/api/src/config/http-app.ts`, `apps/api/src/common/filters/i18n-exception.filter.ts`).
- CORS allows only the origins listed in `CORS_ORIGINS`, with credentials.
- The delivery app pins the API certificate by SHA-256 in release builds (`apps/delivery/lib/app/certificate_pinning.dart`).

---

4. Functional requirements

Each table lists the requirement, its source, the main code that implements it, and its status. Section 4.9 counts the statuses.

4.1 Accounts, sellers and access

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR001	Seller application	A business can apply to sell with business name, contact, phone, email and password. The account starts `PENDING` and cannot sign in. A duplicate email returns 409. Rate limited to 20 per IP per 15 minutes.	P3-Major	DDP-#10, ADR-002	`apps/api/src/sellers/seller.controller.ts`, `seller.service.ts`; `apps/seller/app/onboarding/page.tsx`	Implemented
FR002	Seller approval and account control	An admin lists pending sellers and approves or rejects them (with a reason). An admin can suspend and reinstate an approved seller. Suspended sellers cannot sign in and their products leave the catalogue. Each decision is audited.	P3-Major	DDP-#10, ADR-002	`apps/api/src/sellers/seller-admin.controller.ts`, `seller.service.ts`; `apps/admin/app/sellers/page.tsx`	Implemented
FR003	Customer registration and sign-in	Customers register with name, email and password (at least 8 characters with a letter and a digit). One login endpoint serves all four actor tables. Tokens last 15 days for customers and 8 hours for others. Logout revokes the token.	P1-Blocker	DDP-#04, #07, #08	`apps/api/src/auth/`; `apps/storefront/app/login/page.tsx`; `apps/delivery/lib/screens/login_screen.dart`	Implemented
FR004	Staff and rider account management	An admin creates, edits and deactivates admin, fulfilment-staff and rider accounts. Fulfilment staff can list riders. Changes are audited.	P2-Critical	DDP-#28	`apps/api/src/staff/`, `apps/api/src/delivery/riders-admin.controller.ts`, `riders.service.ts`; `apps/admin/app/staff`, `riders`	Implemented
FR011	Role-based access control	Five roles (CUSTOMER, ADMIN, FULFILMENT_STAFF, SELLER, DELIVERY) are enforced on every route by global guards. Forbidden calls return 403. Services also check ownership of orders, carts, addresses, subscriptions, tickets, products and assignments.	P1-Blocker	DDP-#05, ADR-005	`apps/api/src/auth/guards/`, `auth/decorators/roles.decorator.ts`, `app.module.ts`	Implemented

4.2 Catalogue and seller listings

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR005	Stock-aware catalogue and product page	Anyone can list products with category filter, text search (name and description, case-insensitive) and paging (up to 60 per page). Only active products of approved sellers with at least one valid batch appear. The product page shows images, variants, price, weight, live stock, ingredients, allergens and rating.	P2-Critical	DDP-#11, #19, SDS §5.2, §6.2	`apps/api/src/products/products.controller.ts`, `products.service.ts`, `catalogue.util.ts`; `apps/storefront/app/products/`	Implemented
FR006	Customer favourites	A customer can save products to a favourites list and see it later.	Not set	`FavoriteProduct` model only	`packages/prisma/prisma/schema.prisma` (`FavoriteProduct`). No API route, no service, no screen	Not implemented
FR007	Gift-box bundles	Admins and sellers create bundles of several variants with a price and total weight. Customers browse bundles and add them to the cart. Checkout allocates every component variant by FIFO.	P3-Major	DDP-#12, SDS §1.2, §3.2	`apps/api/src/bundles/`; `apps/storefront/app/bundles/`; `apps/admin/app/bundles`; `apps/seller/app/(portal)/bundles`	Implemented
FR015	Seller catalogue management	A seller creates and edits only their own products, variants and images, and can deactivate them. An admin can do the same for every seller. Images go through presigned upload and magic-byte checks.	P3-Major	DDP-#12, ADR-002, ADR-014	`apps/api/src/products/products-admin.controller.ts`, `products-admin.service.ts`; `apps/seller/app/(portal)/products/`; `apps/admin/app/products/`	Implemented

4.3 Inventory and food safety

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR008	Batch recording and FIFO listing	Admins and owning sellers add a batch per variant with batch number, manufacture date (not in the future), expiry date (after today) and quantity. Batches list in expiry order with colour-coded status (Expired, Expiring soon, Active).	P1-Blocker	DDP-#13, #31, SDS §5.6, §6.6	`apps/api/src/batches/batches.controller.ts`, `batches.service.ts`; `apps/admin/app/products/[id]/batches`; `apps/seller/app/(portal)/products/[id]/batches`	Implemented
FR009	FIFO allocation	Every sale, bundle line and subscription renewal takes stock from the sellable batch with the earliest expiry first, inside the order transaction with `SELECT ... FOR UPDATE`. Short stock returns 409 and nothing is changed. A drained batch becomes `DEPLETED`.	P1-Blocker	DDP-#14, ADR-007, SDS §4.2	`apps/api/src/inventory/inventory.service.ts`	Implemented
FR010	Daily expiry sweep and alerts	Each day at 06:00 UTC, batches past expiry become `EXPIRED` and batches within 7 days become `EXPIRING_SOON`. Each change emails the admins. Expired stock is never sold. One instance runs the sweep per day.	P1-Blocker	DDP-#24, #23, SDS §5.6	`apps/api/src/batches/expiry-cron.service.ts`, `common/locks/cron-lock.service.ts`	Implemented

4.4 Cart, shipping, checkout and payment

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR013	Persisted cart	A signed-in customer's cart (variants and bundles) is stored in the database, so it follows them across devices. Adding or changing a line is checked against valid stock.	P2-Critical	DDP-#15, #20, SDS §5.2	`apps/api/src/cart/`; `apps/storefront/app/cart/page.tsx`	Implemented

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR014	Weight-based shipping	Shipping fee = bracket that contains the total weight (sum of quantity x unit weight). Brackets are cached in Redis for 5 minutes. Admins edit brackets; the service keeps them contiguous and clears the cache.	P2-Critical	DDP-#16, SDS §4.2, §5.3	`apps/api/src/shipping/shipping.service.ts`, `shipping-rates-admin.service.ts`; `apps/admin/app/shipping-rates`	Implemented
FR016	Checkout	The customer picks or adds an address and sees a quote (items, discount, weight, shipping, total) before paying. Checkout runs in one transaction: FIFO decrement, `PENDING_PAYMENT` order with price, weight and fee snapshots, promo redemption, cart emptied.	P1-Blocker	DDP-#17, #20, SDS §5.3, §6.3	`apps/api/src/orders/orders.service.ts`, `orders.controller.ts`, `apps/api/src/addresses/`; `apps/storefront/app/checkout/`	Implemented
FR017	Payments and webhooks	Payment goes through Stripe Checkout or a PayHere form, never through Bio Bloom. Only verified, idempotent webhooks mark orders paid or failed. Failure restocks. Unpaid orders are cancelled after 30 minutes. A late success on a cancelled order is refunded.	P1-Blocker	DDP-#21, #23, ADR-009, ADR-011, ADR-012	`apps/api/src/payments/`, `apps/api/src/orders/stale-order-cron.service.ts`; `apps/storefront/app/checkout/pay`, `checkout/complete`	Implemented
FR018	Promo codes	Admins create percentage or fixed codes with validity dates, minimum order and usage limit. Customers apply one at checkout. The discount never exceeds the item subtotal and never touches shipping. Usage is counted atomically.	P1-Blocker	DDP-#17, SDS §6.3	`apps/api/src/promo-codes/`, `orders.service.ts`; `apps/admin/app/promo-codes`; `apps/storefront/app/checkout/page.tsx`	Implemented

4.5 Subscribe & Save

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR019	Subscription lifecycle	A customer subscribes to a variant every 14, 30 or 60 days at a 10% discount locked at sign-up, after saving a card. No charge is taken at creation; the first billing date is today plus the interval. The customer can pause, resume, skip next and cancel. Admins can list, pause, resume and cancel. The payment token is encrypted with pgcrypto and never returned.	P1-Blocker	DDP-#18, #32, SDS §5.5, §6.4, §7.5	`apps/api/src/subscriptions/subscriptions.service.ts`, `subscriptions-admin.controller.ts`, `token-cipher.service.ts`; `apps/storefront/app/account/subscriptions`; `apps/admin/app/subscriptions`	Implemented
FR020	Recurring billing	Each day at 06:00 UTC, one instance bills every active subscription that is due: FIFO stock, discounted price plus shipping, charge of the stored token, paid order, email. Short stock sends a "stock delayed" email and charges nothing. A declined charge restocks and emails. A cycle key prevents double charges. The billing date always moves forward one interval.	P1-Blocker	DDP-#22, #23, SDS §5.4, ADR-008	`apps/api/src/subscriptions/billing-cron.service.ts`, `common/locks/cron-lock.service.ts`	Implemented

4.6 Orders, fulfilment and delivery

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR022	Order lifecycle, fulfilment and tracking	Orders move only through lawful states: `PENDING_PAYMENT`, `PAID`, `PACKING`, `SHIPPED`, `DELIVERED`, `CANCELLED`, `REFUNDED`. Staff work a queue of open orders. Customers see their order history and progress and can cancel an unpaid order. Admins can cancel paid orders with refund and restock. Each change emails the customer.	P2-Critical	DDP-#26, #31, #32	`apps/api/src/orders/order-status.service.ts`, `orders.controller.ts`, `apps/api/src/fulfilment/`; `apps/storefront/app/account/orders/`; `apps/admin/app/fulfilment`, `orders`	Implemented

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR023	Refunds	A customer requests a full or partial refund on a delivered order, up to the amount still refundable. An admin approves (gateway refund, restock of unexpired batches, order `REFUNDED` when full) or rejects. A gateway failure returns the request to `REQUESTED`. One open request per order.	P2-Critical	DDP-#26	`apps/api/src/refunds/`; `apps/storefront/app/account/orders/[id]/page.tsx`; `apps/admin/app/refund-requests`	Implemented
FR024	Delivery dispatch, GPS and proof of delivery	Staff assign a shipped order to an active rider. The rider moves it `ASSIGNED`, `PICKED_UP`, `OUT_FOR_DELIVERY`, `DELIVERED` or `FAILED`. Each update can carry GPS coordinates captured on the phone. `DELIVERED` needs a proof photo first and closes the order. `FAILED` needs a reason; staff re-open it with an attempt counter.	P2-Critical	DDP-#28, #29, ADR-013, ADR-014	`apps/api/src/delivery/`, `apps/api/src/uploads/`; `apps/delivery/lib/screens/`, `lib/app/location_service.dart`, `upload_service.dart`; `apps/admin/app/deliveries`	Implemented
FR012	Rider push notification	When an order is assigned or re-assigned, the rider gets an FCM push. If Firebase is not configured the API logs and skips; the app still refreshes on pull.	P2-Critical	DDP-#28, #29, ADR-003	`apps/api/src/notifications/push.service.ts`; `apps/delivery/lib/app/push_service.dart`	Implemented

4.7 Customer engagement and notifications

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR025	Moderated reviews	A customer can review a product only if they have a delivered order with it, once per product, rating 1 to 5. Reviews stay hidden until an admin approves. Rejection deletes the review. Approved reviews and the average rating show on the product page.	P4-Minor	DDP-#27	`apps/api/src/reviews/`; `apps/storefront/app/products/[slug]/reviews-section.tsx`; `apps/admin/app/reviews`	Implemented

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR026	Support tickets	A customer opens a ticket (subject and message), lists their own and can reopen a resolved one. A closed ticket cannot be reopened. Admins list, resolve and close tickets. There are no threaded replies.	P4-Minor	DDP-#27	`apps/api/src/support/`; `apps/storefront/app/account/support/page.tsx`; `apps/admin/app/support`	Implemented
FR028	Transactional email	The API emits domain events and sends email without blocking the request. 20 templates cover orders, subscriptions, refunds, batch expiry, seller application results and delivery events. A send failure is logged and returns `{ accepted: false }`.	P2-Critical	DDP-#25, #24, ADR-003	`apps/api/src/notifications/notifications.service.ts`, `mailer.service.ts`, `templates.config.ts`	Partial
FR030	English and Sinhala	The storefront is available in English and Sinhala, chosen with a switcher and kept in the `NEXT_LOCALE` cookie without URL prefixes. `<html lang>` follows the locale. API validation and error messages follow `Accept-Language`, falling back to English.	P2-Critical	DDP-#06, #07, ADR-010	`apps/storefront/i18n/` (`messages/en.json`, `si.json`), `apps/api/src/i18n/`, `common/filters/i18n-exception.filter.ts`	Implemented

4.8 Analytics and seller insight

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR021	Seller sales view	A seller sees paged order lines for their own products only.	P3-Major	DDP-#12	`apps/api/src/sellers/seller-portal.controller.ts`, `seller-portal.service.ts`; `apps/seller/app/(portal)/orders`	Implemented
FR027	Admin analytics	The admin dashboard shows today's revenue, pending orders, active subscriptions and batches expiring within 7 days, a daily revenue series (default 30 days) and top-selling products, counting paid orders only.	P3-Major	DDP-#30, #31, SDS §6.5	`apps/api/src/analytics/`; `apps/admin/app/page.tsx`	Implemented

ID	Title	Description	Priority	Source	Implementing module / path	Status
FR029	Seller dashboard and earnings	A seller sees products on sale, variants low on stock (5 or fewer), batches expiring within 7 days, units sold and revenue this month. Earnings are settled to the seller through payouts.	P3-Major	DDP-#12, DDP-#10 note	`apps/api/src/sellers/seller-portal.service.ts`; `apps/seller/app/(portal)/dashboard/page.tsx`; `Payout` model	Partial

Gaps behind the Partial and Not implemented entries:

- **FR006 Favourites.** Only the `FavoriteProduct` model and its unique index exist. A search of `apps/api/src`, `apps/storefront` and the other apps finds no route, service or screen.
- **FR028 Transactional email.** ADR-003 and `CHANGELOG.md` say seller suspension is emailed, but `DOMAIN_EVENTS` in `apps/api/src/notifications/notifications.service.ts` has no suspension event, and `seller.service.ts` `suspend()` sends none. Emails are English only (`templates.config.ts` header). Events are in-process with no retry, so a crash between commit and send loses the email (noted in the same file).
- **FR029 Payouts.** The dashboard works. The `Payout` model and `Seller.payoutBankAccountEnc` exist, but no code in `apps/api/src` reads or writes payouts. DDP-#10 marks payouts out of scope for these sprints.

Notes on Implemented entries:

- **FR024 GPS.** Coordinates are captured once per status update (`UpdateDeliveryStatusDto.gpsLat`, `gpsLng`). There is no live, continuous tracking of the rider.
- **FR017 live gateways.** Stripe and PayHere code paths exist. The automated tests drive the mock gateway and signed Stripe webhooks; a live sandbox run is not recorded.
- **FR030 scope.** The admin console, seller portal, delivery app and emails are English only by design (ADR-010, DDP-#25).

4.9 Status summary

Status	Count	Requirements
Implemented	27	FR001, FR002, FR003, FR004, FR005, FR007, FR008, FR009, FR010, FR011, FR012, FR013, FR014, FR015, FR016, FR017, FR018, FR019, FR020, FR021, FR022, FR023, FR024, FR025, FR026, FR027, FR030
Partial	2	FR028, FR029
Not implemented	1	FR006
Total	**30**	

4.10 Integration status in the team repositories

The status above is for the integrated code in this repository. On 25 October 2026 the team repositories on GitHub had not yet received all of it:

Repository (`seng34213-biobloom/...`)	Not yet merged to `develop`	Evidence
`backend`	DDP-#16 shipping (PR #60 open), DDP-#10 seller approval (PR #59 open), DDP-#37 OWASP hardening (PR #58 open), DDP-#22 billing job (issue #15 open, no PR). No `bundles`, `promo-codes`, `sellers`, `staff`, `uploads` or `i18n` module in `src/`	PR and issue lists; `develop` tree
`frontend`	Only the storefront exists (catalogue, cart, checkout, subscriptions). No admin console or seller portal. Cross-site sign-in fix PR #21 open	`develop` tree; issue #20
`mobile`	Assignment feed and proof capture (DDP-#29) in PR #4, open. `develop` has the app shell, login and the Android and iOS projects	PR #2 merged, PR #4 open

5. Non-functional requirements

ID	Category	Requirement and target	Evidence in code or measurement	Status
NFR001	Usability	Two-step checkout with the shipping fee shown before payment; S&S self-service with Pause, Resume, Skip next and Cancel; confirmation in an accessible modal before destructive actions (SDS §6.3, §6.4).	`apps/storefront/app/checkout/page.tsx`; `apps/storefront/app/account/subscriptions`; `packages/ui` `Modal`. Browser journey `apps/storefront/e2e/checkout.spec.ts`. No user study was run.	Met
NFR002	Accessibility	WCAG 2.1 Level AA on storefront and admin screens of SDS §6.1 to §6.6. No serious or critical axe violation. Lighthouse accessibility 95 or more.	`docs/wcag-audit.md` reports all criteria conforming after DDP-#36. axe suites exist in `apps/storefront/e2e/a11y.spec.ts`, `apps/admin/e2e/a11y.spec.ts`, `apps/seller/e2e/a11y.spec.ts`. **But** `.github/workflows/ci.yml` has no Playwright job, although the audit says the scan runs on every CI run. Playwright results and Lighthouse scores are not recorded.	Partial
NFR003	Localisation	Storefront UI and API error messages in English and Sinhala; layouts work in both at 320 px.	Catalogues `apps/storefront/i18n/messages/en.json` and `si.json` (486 lines each); `apps/api/src/i18n/translations/`; `i18n.service.spec.ts`. Other apps and emails English only.	Met

ID	Category	Requirement and target	Evidence in code or measurement	Status
NFR004	Security: authentication	bcrypt cost 12; HS256 JWT with a secret of at least 32 characters; 15-day customer and 8-hour staff tokens; `HttpOnly`, `SameSite=Strict`, `Secure` cookie in production; revocation by Redis JTI blocklist (SDS §5.1, §7.1).	`apps/api/src/auth/auth.service.ts` (`BCRYPT_COST`, `issueToken`), `auth.module.ts`, `strategies/jwt.strategy.ts`, `auth.controller.ts`; `config/env.validation.ts`. Tests: `auth.service.spec.ts`, `jwt.strategy.spec.ts`, `test/auth.e2e-spec.ts`.	Met
NFR005	Security: authorisation	403 for any role or ownership violation; customers cannot escalate privileges (SDS §7.2).	Global guards in `apps/api/src/app.module.ts`; separate actor tables (ADR-005). Tests: `roles.guard.spec.ts`, `jwt-auth.guard.spec.ts`, `test/ownership.e2e-spec.ts` (5 cases), `test/seller.e2e-spec.ts`.	Met
NFR006	Security: input and injection	DTO validation rejects bad input with 422 and unknown fields; no SQL string building (SDS §7.4).	`ValidationPipe` with `whitelist`, `forbidNonWhitelisted`, `errorHttpStatusCode: 422` in `apps/api/src/config/http-app.ts`. `$queryRawUnsafe` and `$executeRawUnsafe` do not appear in `apps/api/src`. Raw SQL uses tagged templates in four files only: `health.controller.ts`, `token-cipher.service.ts`, `promo-codes.service.ts`, `inventory.service.ts`.	Met
NFR007	Security: abuse protection	20 requests per IP per 15 minutes on login, register and seller apply; 100 requests per user per minute elsewhere; counters shared across instances; double-submit CSRF on every mutation (SDS §7.4).	`apps/api/src/common/guards/throttler.config.ts` (`SPEC_THROTTLE_LIMITS`), `user-throttler.guard.ts`, Redis storage in `app.module.ts`; `common/middleware/csrf.middleware.ts`. Tests: `user-throttler.guard.spec.ts`, `csrf.middleware.spec.ts`, `test/auth.e2e-spec.ts`.	Met

ID	Category	Requirement and target	Evidence in code or measurement	Status
NFR008	Security: data protection	No card data stored; S&S tokens encrypted at rest; HSTS and CSP; uploads checked by magic bytes, 5 MB cap; TLS 1.3 in transit; keys in a managed secret store (SDS §7.3, §7.4).	Met: tokens via pgcrypto (`token-cipher.service.ts`), only gateway ids in `Payment`; helmet CSP and HSTS (`http-app.ts`); CSP headers in each `next.config.mjs`; upload checks (`uploads.service.ts`). **Not as specified:** the Terraform ALB policy is `ELBSecurityPolicy-TLS13-1-2-2021-06` (`infra/terraform/alb.tf:48`), which accepts TLS 1.2 as well as 1.3, and CloudFront allows `TLSv1.2_2021` (`infra/terraform/storage.tf:72`). That Terraform is not applied; staging TLS is managed by Render and Vercel and its version was not measured. SSM and KMS are not used on Render.	Partial
NFR009	Portability: mobile	The delivery app runs on Android 10 (API 29) or later and iOS 13 or later.	Team repository `seng34213-biobloom/mobile`, branch `develop`: `apps/delivery/android/app/build.gradle.kts` line 27 `minSdk = 29`; `apps/delivery/ios/Runner.xcodeproj/project.pbxproj` `IPHONEOS_DEPLOYMENT_TARGET = 13.0` (lines 349, 475, 526); `ios/Flutter/AppFrameworkInfo.plist` `MinimumOSVersion` 13.0 (merged PR #2). This repository has no `apps/delivery/android` or `ios` folder; `apps/delivery/README.md` gives the same settings to apply after `flutter create`. Not tested on physical devices.	Met
NFR010	Performance: checkout	95th-percentile checkout latency under 600 ms with 50 concurrent shoppers for two minutes; fewer than 1% failed requests.	Thresholds in `infra/load-test/k6/checkout.js` (`p(95)<600`, `rate<0.01`). **Source:** DDP-#38 and `infra/load-test/README.md`. SDS v01 states no latency figure (DDP-#38 says so too). No k6 run is recorded.	Not measured
NFR011	Performance: billing job	One billing run processes 200 due subscriptions in under 60 seconds, each exactly once, with no negative stock.	`infra/load-test/k6/subscription-cron.js` (`billing_run_ms max<60000`), fixture `infra/load-test/fixtures/due-subscriptions.sql`, trigger `POST /api/admin/subscriptions/billing/run` (404 in production). No run is recorded.	Not measured
NFR012	Scalability	The API is stateless and can run as 1 to 3 instances, scaling at 70% CPU (SDS §8.1).	State lives in PostgreSQL and Redis; cron locks and Redis throttling make several instances safe. Autoscaling exists only in `infra/terraform/ecs.tf`, which is not applied. Not load-tested.	Partial

ID	Category	Requirement and target	Evidence in code or measurement	Status
NFR013	Reliability	Payments and billing never double-charge; stock never goes negative; email failure never fails a request; health endpoints for the platform.	Idempotent webhooks and `Payment.billingCycleKey` unique (`payments.service.ts`, `schema.prisma`); cron locks (`cron-lock.service.ts`); stale-order job; `GET /api/health` and `/api/health/ready`; graceful shutdown in `main.ts`. Tests: `test/webhook-signature.e2e-spec.ts`, `test/checkout.e2e-spec.ts` (concurrent last unit), `billing-cron.service.spec.ts`, `cron-lock.service.spec.ts`.	Met
NFR014	Data integrity	Database CHECK constraints for stock, ratings, quantities, refunds and promo usage; exact decimal money; UTC dates; price, weight and fee snapshots on orders (SDS §3.3).	`packages/prisma/prisma/migrations/20261022090000_spec_completion/migration.sql`; `OrderItem.unitPriceSnapshot`, `Order.shippingFee`, `Order.totalWeight` in `schema.prisma`; `process.env.TZ = 'UTC'` in `apps/api/src/main.ts`.	Met
NFR015	Maintainability	Lint with zero warnings, typecheck and Prettier on every pull request; shared types between API and web apps; decisions recorded as ADRs; endpoints documented.	`.github/workflows/ci.yml` (lint, typecheck, format, actionlint); `packages/shared-types`; `docs/adr/` (14 ADRs); `docs/api-reference.md`.	Met
NFR016	Testability	API unit coverage at or above the Jest gate of 65% statements, 50% branches, 65% functions, 65% lines (`apps/api/jest.config.json`; SDS §8.2 says 65%). The Definition of Done asks for 80%.	Measured 25 October 2026: 24 suites, 231 tests, all pass. Coverage 31.24% statements (1198/3834), 23.38% branches (348/1488), 21.82% functions (163/747), 31.54% lines (1085/3440). Modules at 0%: addresses, analytics, bundles, cart, fulfilment, health, products, promo-codes, refunds, staff. Web Vitest: storefront 11 tests, admin 7, seller 2, `packages/ui` 20, all pass. API e2e (11 suites) and Flutter tests (21 cases in 6 files) were not run for this document.	Not met
NFR017	Security: dependencies	Zero high or critical vulnerabilities in `pnpm audit` (SDS §8.2 stage 5).	Measured 25 October 2026: 163 vulnerabilities (15 low, 69 moderate, 69 high, 10 critical); exit code 1. Critical advisories include `next` (authorization bypass in middleware, fixed in 14.2.25; the apps use 14.2.5), `handlebars`, `tinypool`, `proxy-addr` and `vitest`.	Not met

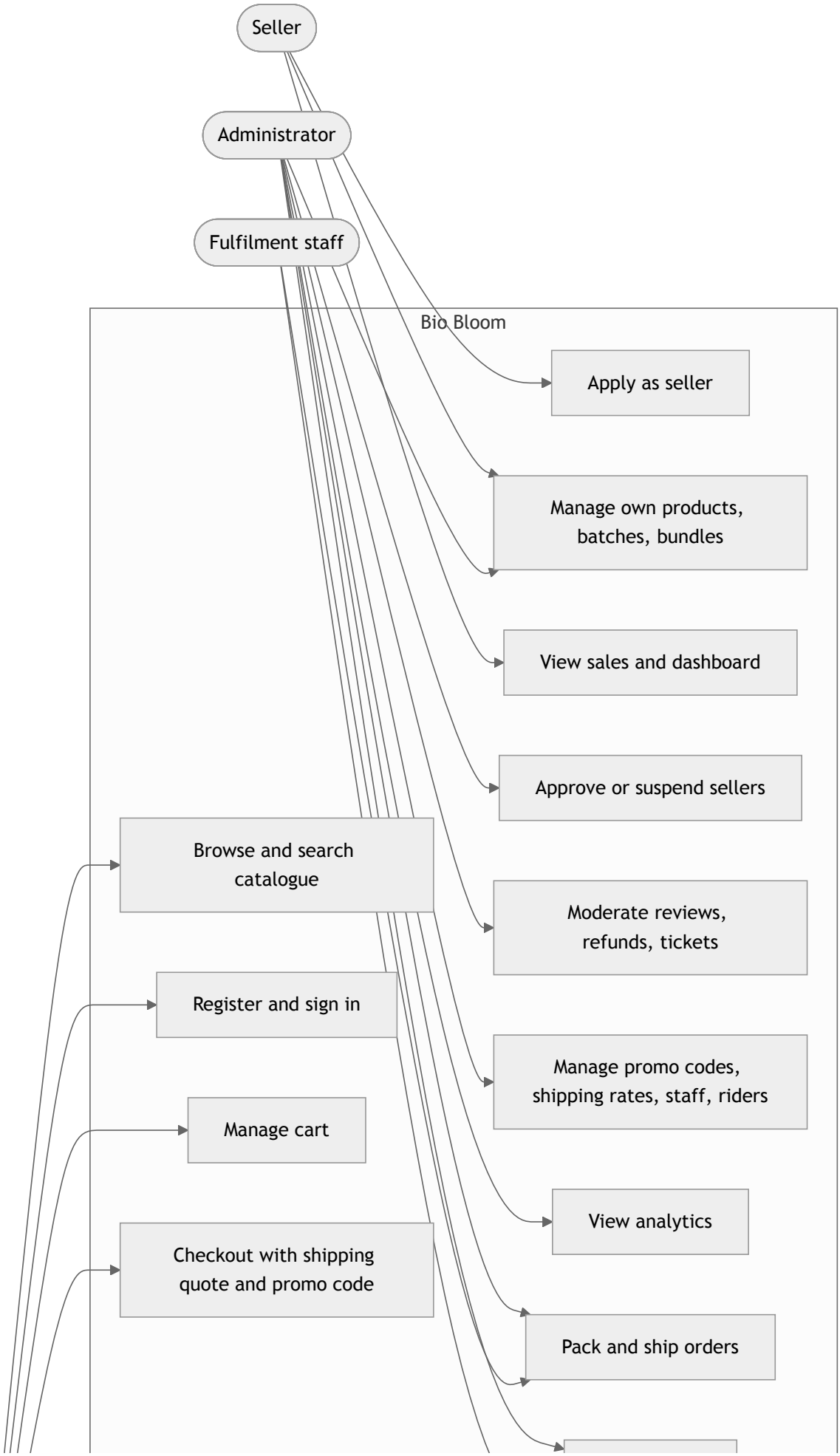
ID	Category	Requirement and target	Evidence in code or measurement	Status
NFR018	Auditability	Every admin, seller, rider and system mutation is written to an audit log with the real actor (SDS §7.2 AdminActionLog).	`AuditLog` model; `AuditService.record` is called from 14 services: batches, bundles, delivery, riders, order status, products admin, promo codes, refunds, reviews, sellers, shipping rates admin, staff, subscriptions and support. There is no screen or endpoint to read the log.	Met
NFR019	Deployability and availability	Automated deploy to staging from `develop` after CI, with a health check and rollback; manual approval for production (SDS §8.2 stages 7 and 8).	Monorepo `.github/workflows/deploy.yml` targets ECS with health check and rollback, but AWS is not provisioned. Real staging: Render deploy after CI passes (team backend PR #51) and Vercel for the storefront. No production environment exists. Uptime was not measured.	Partial
NFR020	Security: least privilege for billing	The billing job runs under a dedicated database account that can only read subscriptions and write orders (SDS §7.5).	The job runs inside the API process with the API's single `DATABASE_URL`. No `CREATE ROLE` or `GRANT` exists in `packages/prisma/prisma/migrations` or `infra/`.	Not met

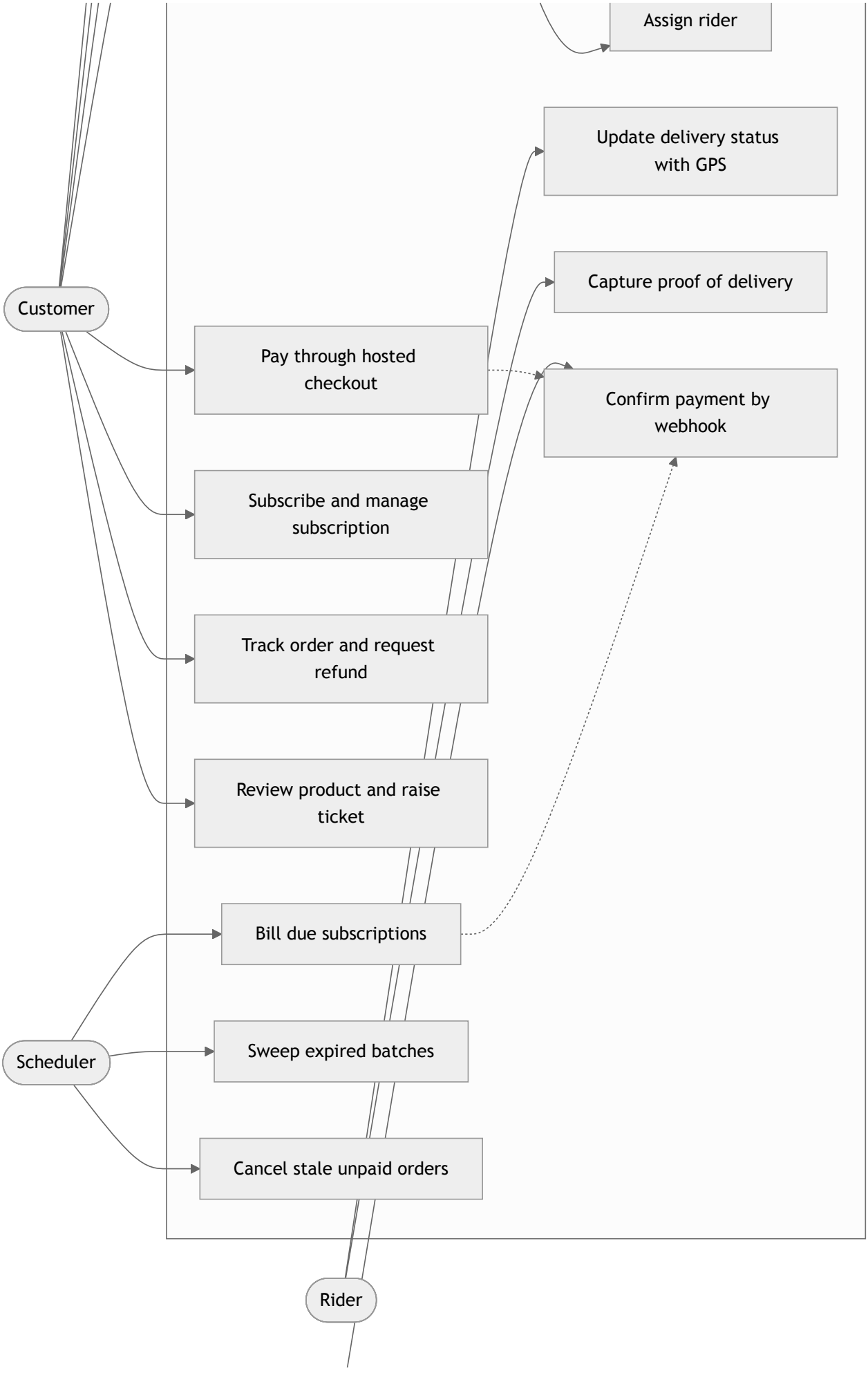
NFR status summary: Met 11 (NFR001, NFR003, NFR004, NFR005, NFR006, NFR007, NFR009, NFR013, NFR014, NFR015, NFR018); Partial 4 (NFR002, NFR008, NFR012, NFR019); Not met 3 (NFR016, NFR017, NFR020); Not measured 2 (NFR010, NFR011).

Other SDS targets that were not measured: the SDS §7.5 rule that a cancellation email arrives within 60 seconds has no test or measurement.

---

6. Use-case overview


Seller
Administrator
Fulfilment staff
Bio Bloom
Apply as seller
Manage own products, batches, bundles
View sales and dashboard
Approve or suspend sellers
Browse and search catalogue
Moderate reviews, refunds, tickets
Register and sign in
Manage promo codes, shipping rates, staff, riders
Manage cart
View analytics
Checkout with shipping quote and promo code
Pack and ship orders


Assign rider
Update delivery status with GPS
Capture proof of delivery
Customer
Pay through hosted checkout
Confirm payment by webhook
Subscribe and manage subscription
Track order and request refund
Review product and raise ticket
Bill due subscriptions
Scheduler
Sweep expired batches
Cancel stale unpaid orders
Rider



Use case	Functional requirements
UC1	FR005, FR007
UC2	FR003, FR011, FR030
UC3	FR013
UC4	FR014, FR016, FR018
UC5, UC23	FR017
UC6, UC20	FR019, FR020
UC7	FR022, FR023
UC8	FR025, FR026
UC9, UC12	FR001, FR002
UC10	FR007, FR008, FR015
UC11	FR021, FR029
UC13	FR023, FR025, FR026
UC14	FR004, FR014, FR018
UC15	FR027
UC16, UC17	FR022, FR024, FR012
UC18, UC19	FR024
UC21	FR010, FR028
UC22	FR017

---

7. Traceability matrix

User stories are quoted (shortened) from `docs/development-issues.md`. API e2e files are in `apps/api/test/`; unit specs are next to the code in `apps/api/src/`. "None" means no automated test was found for that requirement.

7.1 Functional user stories

DDP	User story	Requirements	Tests
DDP-#04	As a customer, I want to register and sign in securely so that I can keep a cart, place orders and manage my subscriptions.	FR003	`auth.e2e-spec.ts`; `auth/auth.service.spec.ts`; `auth/strategies/jwt.strategy.spec.ts`

DDP	User story	Requirements	Tests
DDP-#05	As a system administrator, I want every API route protected by role checks, revocable sessions, CSRF protection and rate limits ...	FR011, NFR005, NFR007	`auth.e2e-spec.ts`; `ownership.e2e-spec.ts`; `auth/guards/roles.guard.spec.ts`; `auth/guards/jwt-auth.guard.spec.ts`; `common/middleware/csrf.middleware.spec.ts`; `common/guards/user-throttler.guard.spec.ts`
DDP-#06	As a client-app developer, I want consistent validation errors, predictable CORS behaviour and a health endpoint ...	FR030, NFR006, NFR013	`auth.e2e-spec.ts` (422, health); `i18n/i18n.service.spec.ts`; `config/env.validation.spec.ts`; `common/middleware/http-request.middleware.spec.ts`
DDP-#07	As a customer, I want a branded, accessible storefront where I can sign in or register and choose English or Sinhala ...	FR003, FR030, NFR002	`apps/storefront/e2e/a11y.spec.ts` (lang="si"); `apps/storefront/lib/api.test.ts`; `apps/storefront/lib/navigation.test.ts`; `packages/ui/src/components/__tests__/*.test.tsx`
DDP-#08	As a delivery rider, I want to sign in to the mobile app once per shift ...	FR003, NFR009	`apps/delivery/test/login_screen_test.dart`; `auth_store_test.dart`; `api_client_test.dart`
DDP-#10	As an artisan seller, I want to apply for a seller account and be told when it is approved ...	FR001, FR002	`seller.e2e-spec.ts`; `sellers/seller.service.spec.ts`
DDP-#11	As a shopper, I want to browse and search only products that are actually in stock ...	FR005	`fifo.e2e-spec.ts` (`GET /api/products` excludes zero stock). Products module unit coverage is 0%
DDP-#12	As a seller, I want to create and maintain my own products and variants ...	FR015, FR007, FR021, FR029	`apps/seller/e2e/products.spec.ts`; `uploads/uploads.service.spec.ts`; `apps/seller/lib/guard.test.ts`. None for bundles or seller portal service
DDP-#13	As an admin or seller, I want to record each production batch with its manufacture and expiry dates ...	FR008	`apps/admin/e2e/batches.spec.ts`; `apps/seller/e2e/products.spec.ts`; `apps/admin/components/BatchStatusBadge.test.tsx`
DDP-#14	As the business owner, I want every sale to draw from the batch that expires soonest ...	FR009	`fifo.e2e-spec.ts`; `checkout.e2e-spec.ts` (concurrent last unit); `inventory/inventory.service.spec.ts`
DDP-#15	As a customer, I want my cart saved to my account and checked against real stock ...	FR013	`checkout.e2e-spec.ts`; `ownership.e2e-spec.ts` (cart line). Cart module unit coverage is 0%
DDP-#16	As a customer, I want the shipping fee calculated from the real weight of my order ...	FR014	`shipping-quote.e2e-spec.ts` (500 g and 501 g); `shipping/shipping.service.spec.ts`; `shipping/shipping-rates-admin.service.spec.ts`
DDP-#17	As a customer, I want to see my total with shipping and place my order in one reliable step ...	FR016, FR018	`checkout.e2e-spec.ts`; `ownership.e2e-spec.ts` (address). None for promo codes

DDP	User story	Requirements	Tests
DDP-#18	As a customer, I want to subscribe to a product at a discount and pause, skip or cancel whenever I like ...	FR019	`subscriptions/subscriptions.service.spec.ts`; `subscription-cron.e2e-spec.ts` (pgcrypto round trip); `ownership.e2e-spec.ts`; `apps/storefront/lib/subscriptions.test.ts`
DDP-#19	As a shopper, I want to browse the catalogue and see each product's sizes, price, weight and availability ...	FR005	`apps/storefront/components/stock-indicator.test.tsx`; `apps/storefront/app/products/[slug]/variant-panel.test.tsx`; `apps/storefront/e2e/a11y.spec.ts`
DDP-#20	As a customer, I want to review my cart with the shipping fee shown up front and then pay in a clear second step ...	FR013, FR016, NFR001	`apps/storefront/e2e/checkout.spec.ts`
DDP-#21	As a customer, I want to pay through a trusted hosted checkout and have my order confirmed automatically ...	FR017	`webhook-signature.e2e-spec.ts`; `checkout.e2e-spec.ts`; `payments/payments.service.spec.ts`
DDP-#22	As a subscriber, I want my recurring order charged and created automatically on its due date ...	FR020	`subscription-cron.e2e-spec.ts`; `subscriptions/billing-cron.service.spec.ts`; k6 `infra/load-test/k6/subscription-cron.js` (not run)
DDP-#23	As a subscriber, I want to be charged exactly once per billing cycle ...	FR020, FR017, NFR013	`subscriptions/billing-cron.service.spec.ts`; `common/locks/cron-lock.service.spec.ts`; `webhook-signature.e2e-spec.ts` (duplicates)
DDP-#24	As an admin, I want to be alerted when a batch is within seven days of expiry and have expired stock withdrawn ...	FR010, FR028	`batches/expiry-cron.service.spec.ts`; `fifo.e2e-spec.ts` (expired batch never touched)
DDP-#25	As a customer, I want an email whenever my order, subscription or refund changes state ...	FR028	`notifications.e2e-spec.ts` (8 of the 20 templates); `notifications/mailer.service.spec.ts`
DDP-#26	As a fulfilment staff member, I want a queue of paid orders that I can move through packing, shipping and delivery ...	FR022, FR023	`orders/order-status.service.spec.ts`; `refund.e2e-spec.ts`; `ownership.e2e-spec.ts` (order). Fulfilment and refunds unit coverage is 0%
DDP-#27	As a customer, I want to review products I have received and raise a support ticket ...	FR025, FR026	`reviews/reviews.service.spec.ts`; `support/support.service.spec.ts`; `ownership.e2e-spec.ts` (ticket)
DDP-#28	As a delivery rider, I want to see the orders assigned to me and record pickup, delivery and proof ...	FR024, FR004, FR012	`delivery.e2e-spec.ts`; `delivery/delivery.service.spec.ts`; `uploads/uploads.service.spec.ts`. None for staff and rider management

DDP	User story	Requirements	Tests
DDP-#29	As a delivery rider, I want my phone to show my deliveries, notify me of new ones and let me confirm each drop with a photo ...	FR024, FR012, NFR009	`apps/delivery/test/assignments_feed_screen_test.dart`; `proof_of_delivery_screen_test.dart`; `upload_service_test.dart`. None for push
DDP-#30	As an admin, I want today's revenue, pending orders, active subscriptions and expiring batches in one call ...	FR027	None (analytics unit coverage 0%)
DDP-#31	As an admin, I want a dashboard that highlights batches near expiry and a screen to manage each product's batches ...	FR027, FR008	`apps/admin/e2e/batches.spec.ts`; `apps/admin/lib/nav.test.ts`; `apps/admin/e2e/a11y.spec.ts`
DDP-#32	As a customer, I want one place to manage my subscriptions and follow my orders ...	FR019, FR022	`apps/storefront/lib/subscriptions.test.ts`
None	No user story in the backlog	FR006	None

7.2 Quality and enabling user stories

DDP	User story	Requirements	Tests or checks
DDP-#01	As a developer, I want to clone the repository and start every app and backing service with a few documented commands ...	NFR015	`docker-compose.yml`; `README.md` quick start
DDP-#02	As a team member, I want agreed branching, review and documentation conventions ...	NFR015	`CONTRIBUTING.md`; `docs/adr/`; `CHANGELOG.md`
DDP-#03	As a backend developer, I want the full relational schema, a migration and repeatable seed data ...	NFR014 (all FRs)	`packages/prisma/prisma/migrations/`; `seed.ts`; CI seeds before the e2e job
DDP-#09	As a reviewer, I want every pull request linted, type-checked and format-checked automatically ...	NFR015	`.github/workflows/ci.yml`
DDP-#33	As a developer, I want every service module covered by unit tests with an enforced threshold ...	NFR016	`apps/api/jest.config.json` gate; measured coverage in NFR016 (not met)
DDP-#34	As a release manager, I want the end-to-end API flows verified against a real database on every pull request ...	NFR016, NFR013	CI job "API integration tests" in `.github/workflows/ci.yml`
DDP-#35	As a QA engineer, I want automated tests for the main customer and admin journeys ...	NFR016, NFR002	`apps/*/e2e/*.spec.ts` (Playwright); not wired into CI
DDP-#36	As a customer who relies on a keyboard or screen reader, I want every screen to meet WCAG 2.1 AA ...	NFR002	`docs/wcag-audit.md`; `apps/*/e2e/a11y.spec.ts`
DDP-#37	As the business owner, I want the platform hardened against the OWASP Top Ten ...	NFR004 to NFR008	`config/env.validation.spec.ts`; `uploads/uploads.service.spec.ts`; `ownership.e2e-spec.ts`

DDP	User story	Requirements	Tests or checks
DDP-#38	As the business owner, I want proof that checkout stays fast under load and that the daily billing run finishes quickly ...	NFR010, NFR011	`infra/load-test/k6/checkout.js`; `subscription-cron.js` (not run)
DDP-#39	As a release manager, I want every merge tested, audited and packaged as a versioned container image ...	NFR015, NFR017	`.github/workflows/ci.yml` (audit, Docker build, ECR push)
DDP-#40	As the project team, I want a staging environment that deploys itself from `develop` and rolls back on failure ...	NFR019	`.github/workflows/deploy.yml`; `infra/terraform/`; Render deploy (team backend PR #51)

7.3 Coverage of requirements by tests

Requirements with no automated test found	Reason
FR004, FR007, FR012, FR018, FR021, FR027, FR029	No spec or e2e file exercises the module
FR006	Not implemented

---

8. Open gaps and follow-up checklist

- [ ] FR006: build favourites (API routes, storefront button and list) or remove the `FavoriteProduct` model.
- [ ] FR028: add a seller-suspended event and email, or correct ADR-003 and `CHANGELOG.md`.
- [ ] FR029: implement payout calculation and settlement, or record it as out of scope in an ADR.
- [ ] SDS §6.1 newsletter: `apps/storefront/components/newsletter-form.tsx` only confirms locally ("There is no subscription endpoint yet").
- [ ] NFR002: add a Playwright job to `.github/workflows/ci.yml`, or correct the claim in `docs/wcag-audit.md`; record Lighthouse scores.
- [ ] NFR008: decide whether TLS 1.3 only is required. If so, use a TLS 1.3-only ALB policy and record the staging TLS version.
- [ ] NFR010, NFR011: run both k6 scripts and record the results.
- [ ] NFR016: add unit tests for the 0% modules (products, cart, bundles, promo codes, refunds, analytics, fulfilment, staff, addresses, health) until the 65/50/65/65 gate passes.
- [ ] NFR017: upgrade `next` to 14.2.25 or later and clear the other high and critical advisories.
- [ ] NFR020: create a restricted database role for the billing job, or record the decision not to.
- [ ] Merge the open team pull requests listed in Section 4.10 so the team repositories match this specification.
- [ ] Fix cross-site sign-in on staging (team frontend issue #20).

Appendix A: Data model summary

`packages/prisma/prisma/schema.prisma` defines **25 models**. The SDS listed 14; the schema header counts a 9-entity marketplace extension plus `CartItem`, and `ProductImage` was added later, which gives 25.

Group	Models	Origin
SDS §3.2 entities (14)	Customer, Product, ProductVariant, ProductBatch, Bundle, BundleItem, Order, OrderItem, Subscription, ShippingRate, Review, Address, Payment, AdminUser	SDS v01
Marketplace and operations (9)	Seller, Payout, PromoCode, RefundRequest, FavoriteProduct, AuditLog, DeliveryUser, DeliveryAssignment, SupportTicket	ADR-002, ADR-005
Supporting (2)	CartItem (persisted cart, SDS §5.2), ProductImage (gallery, SDS §6.2)	SDS sequences and wireframes

Enums: `Role`, `AdminRole`, `OrderStatus`, `PaymentStatus`, `SubscriptionStatus`, `BatchStatus`, `SellerStatus`, `DeliveryAssignmentStatus`, `PaymentGateway`, `DiscountType`, `PayoutStatus`, `RefundStatus`, `SupportTicketStatus`, `AuditActorType`.

Appendix B: Business rules enforced in code

ADR-002 cites `BR001` and `BR004` for the seller approval rules; their v01 wording is not available, so they are mapped to FR001 and FR002 here. The rules below are read from the code.

Rule	Value	Where
Sell from the batch with the earliest expiry date first	FIFO	`apps/api/src/inventory/inventory.service.ts`
A batch counts as "expiring soon"	7 days or fewer before expiry	`apps/api/src/batches/expiry-cron.service.ts`, `batches.service.ts`
Daily jobs (billing, expiry sweep)	06:00 UTC	`CronExpression.EVERY_DAY_AT_6AM`
Unpaid order cancelled after	30 minutes (configurable)	ADR-011; `apps/api/src/orders/stale-order-cron.service.ts` (runs every 5 minutes)
S&S intervals	14, 30 or 60 days	`apps/api/src/subscriptions/dto/`
S&S discount, locked at sign-up	10%	`DEFAULT_DISCOUNT_RATE = '0.1000'` in `subscriptions.service.ts`
Order total	items − discount + shipping	`apps/api/src/orders/orders.service.ts`
Promo discount limit	Never above item subtotal; never on shipping	`apps/api/src/promo-codes/promo-codes.service.ts`
Shipping-rate cache	5 minutes in Redis	`apps/api/src/shipping/shipping.service.ts`

Rule	Value	Where
Review allowed only after a delivered order; one per customer and product; rating	1 to 5	`apps/api/src/reviews/reviews.service.ts` ; CHECK in migration
Refund amount	Not above the refundable amount	`apps/api/src/refunds/refunds.service.ts` ; CHECK in migration
Upload types and size	JPEG, PNG, WebP, HEIC; 5 MB	`apps/api/src/uploads/dto/presign.dto.ts` , `uploads.service.ts`
Seller low-stock threshold on dashboard	5 units or fewer	`LOW_STOCK_THRESHOLD` in `apps/api/src/sellers/seller-portal.service.ts`
Product list page size	Default 24, maximum 60	`apps/api/src/products/dto/list-products-query.dto.ts`

End of document.